Power Transformer

Module 31 Study Notes • Feature Engineering

1. Introduction to Power Transforms

In the previous module, we learned how to manually apply specific mathematical functions (like Log or Square Root) using the `FunctionTransformer` to force skewed data into a **Normal Distribution**. The goal remains the same: optimizing the performance of linear machine learning algorithms (like Linear Regression or Logistic Regression).

However, manually guessing whether to use Log, Square, or Square Root for every single column is tedious and inefficient. Enter the **Power Transformer**. A Power Transform automatically searches for the optimal mathematical function to apply to each individual column to make it as normally distributed as mathematically possible.

2. The Architecture: Box-Cox & Yeo-Johnson

Scikit-Learn provides two primary algorithms within the `PowerTransformer` class to achieve this optimization.

A. Box-Cox Transform

The Box-Cox transform searches for an optimal exponent (represented by the Greek letter Lambda, λ) between -5 and 5. It tests raising the data to these different powers (e.g., $x^0.5$ or x^-1) and selects the λ that yields the most normal distribution.

> **Crucial Limitation:** The Box-Cox transform **strictly requires strictly positive data**. It will fail completely if a column contains zero (0) or any negative numbers.

B. Yeo-Johnson Transform (The Default)

The Yeo-Johnson transform is a modern mathematical adjustment to the Box-Cox method. It serves the exact same purpose (finding the optimal λ), but its mathematical formula is adjusted to seamlessly handle zeroes and negative numbers.

> **Standardization Bonus:** By default, the `PowerTransformer` automatically applies zero-mean, unit-variance standardization to the transformed output (similar to `StandardScaler`). You do not need to scale the data separately!

3. Implementation Workflow

The `PowerTransformer` is designed to be easily plugged into a ColumnTransformer or Pipeline.

```
from sklearn.preprocessing import PowerTransformer

# 1. Initialize the transformer
# Note: method='yeo-johnson' is the default and safest choice.
# method='box-cox' can be explicitly passed if your data is strictly > 0.
pt = PowerTransformer(method='yeo-johnson', standardize=True)

# 2. Fit and Transform the Training Data
# The transformer calculates the optimal lambda for EACH column individually.
X_train_transformed = pt.fit_transform(X_train)

# 3. Transform the Test Data
# It applies the same learned lambda values to the test set to prevent data
leakage.
X_test_transformed = pt.transform(X_test)
```

4. Peeking Under the Hood

After you fit the `PowerTransformer` on your training data, you can actually see what exponents (λ) the algorithm decided were optimal for each of your columns. This is excellent for debugging and understanding your data's shape.

```
# Assuming X_train had 3 columns, this will return an array of 3 numbers.
print(pt.lambdas_)

# Output example: [ 0.17  -0.54   1.21 ]
# This means:
# Column 1 was raised to the power of 0.17
# Column 2 was raised to the power of -0.54 (a reciprocal-like transform)
```

```
# Column 3 was raised to the power of 1.21
```

5. Key Takeaways

- If you are using a linear model (Linear Regression, Logistic Regression, SVMs) and your data is heavily skewed, `PowerTransformer` is often the most powerful automated tool to fix the distribution.
- Always default to **Yeo-Johnson** unless you are 100% certain your entire dataset contains strictly positive values without zeroes.
- Remember that this transformation includes automatic scaling (Standardization).